Clang-Tidy: Modernize

Giuseppe Di Blasi
Università di Parma

8 Gennaio 2026

Cos'è Clang-Tidy

Clang-Tidy

- Strumento di linting basato su Clang per C++
- Offre un framework estensibile
- Individua e correggere :
  - Errori di programmazione
  - Cattive pratiche di programmazione
  - Problematiche di stile

In questo seminario ci concentreremo sulla parte di modernize, quindi la rilevazione di codice legacy e gli opportuni suggerimenti per adeguarlo allo standard moderno.

TinyXML2

TinyXML2

- Parser per file XML scritto in C++
- Progetto abbastanza grande
- Presenta codice legacy

Clang-Tidy modernize

- Comando: **clang-tidy tinyxml2.cpp -checks="modernize-*" -- -std=c++17**
- C++ 17 come standard target
- 1700 warning generati

Risultato esecuzione

- 111 modernize-use-trailing-return-type
- 103 modernize-use-nullptr
- 35 modernize-avoid-c-arrays
- 11 modernize-use-auto
- 3 modernize-use-equals-default
- 3 modernize-loop-convert
- 1 modernize-raw-string-literal

modernize-use-trailing-return-type

Codice legacy

```
char* StrPair::ParseText( char* p, const char* endTag, int strFlags, int*
    curLineNumPtr ){
  //...
}
```

Codice moderno

```
auto StrPair::ParseText( char* p, const char* endTag, int strFlags, int*
    curLineNumPtr ) -> char*{
  //...
}
```

modernize-use-nullptr

Codice legacy

```
//const char* XMLUtil::GetCharacterRef(const char* p, char* value, int* length)
const char* q = 0; // riga 506

// length_ptr sarebbe più appropriato
if (length == 0) // riga 569
```

Codice moderno

```
const char* q = nullptr;

// length_ptr sarebbe più appropriato
if (length == nullptr);
```

modernize-use-auto

Codice legacy

```
const unsigned char* pu = reinterpret_cast<const unsigned char*>(p);

XMLText* text = CreateUnlinkedNode<XMLText>( _textPool );
```

Codice moderno

```
auto pu = reinterpret_cast<const unsigned char*>(p);

auto text = CreateUnlinkedNode<XMLText>( _textPool );
```

modernize-raw-string-literal

Codice legacy

```
XMLDeclaration* XMLDocument::NewDeclaration( const char* str ){
  XMLDeclaration* dec = CreateUnlinkedNode<XMLDeclaration>( _commentPool );
  dec->SetValue( str ? str : "xml version=\"1.0\" encoding=\"UTF-8\"" );
  return dec;
}
```

Codice moderno

```
XMLDeclaration* XMLDocument::NewDeclaration( const char* str ){
  XMLDeclaration* dec = CreateUnlinkedNode<XMLDeclaration>( _commentPool );
  dec->SetValue( str ? str : R"( xml version="1.0" encoding="UTF-8" )" );
  return dec;
}
```

modernize-avoid-c-arrays

Codice legacy

```
static const char* TRUE_VALS[] = { "true", "True", "TRUE", 0 };
static const char* FALSE_VALS[] = { "false", "False", "FALSE", 0 };
```

Codice moderno

```
static constexpr std::array<const char*, 3> TRUE_VALS = {
  "true", "True", "TRUE"
};

static constexpr std::array<const char*, 3> FALSE_VALS = {
  "false", "False", "FALSE"
};
```

modernize-loop-convert

Codice legacy

```
for( int i=0; i<NUM_ENTITIES; ++i ) {
  if ( entities[i].value == *q ) {
    //..
    Write( entities[i].pattern, entities[i].length );
  }
}
```

Codice moderno

```
  //si potrebbe usare const auto& per efficienza...
  for(auto entitie : entities) {
    if ( entitie.value == *q ) {
      //..
      Write( entitie.pattern, entitie.length );
    }
  }
```

modernize-use-equals-default

Codice legacy

```
XMLComment::~XMLComment() { }
XMLUnknown::~XMLUnknown() { }
```

Codice moderno

```
XMLComment::~XMLComment() = default;
XMLUnknown::~XMLUnknown() = default;
```

Conclusioni

- Aggiornato TinyXML2 allo standard C++ 17
- Facilitato l'aggiornamento allo standard moderno
- Migliorato la sicurezza del programma (nullptr, std::array)
- Migliorato la comprensione del codice con costrutti di sintassi moderna
- Alcuni check non fanno autofix: modernize-avoid-c-arrays

Riferimenti

- GitHub TinyXML2: `https://github.com/leethomason/tinyxml2`
- Documentazione Clang-Tidy: `https://clang.llvm.org/extra/clang-tidy`
- C++ Core guidelines:
  `https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines.html`